



DOCUMENTO 09

Documentación Técnica

Estructura, tecnologías, configuración, ejecución local y despliegue

Proyecto	PARQUEADERO — Sistema de gestión de parqueadero
Institución	I.E.R. Santa María
Integrantes	Loren
Docente	Liseth Giraldo
Versión / Fecha	1.0 — 1 de octubre de 2026

Documentación Técnica

Alcance de este documento y fuentes de información

Este documento distingue tres tipos de información:

- **EXISTENTE** Elementos verificados en el repositorio del proyecto a la fecha de elaboración.
- **PROPUESTO** Diseño planteado para el sistema de parqueadero, aún no implementado en el código.
- **PENDIENTE** Datos que deben ser completados por el equipo del proyecto.

El video de referencia corresponde a una sesión de clase que muestra la **metodología y las herramientas** de desarrollo (repositorio en GitHub, frontend PWA con Vite, base de datos MySQL en Clever Cloud y despliegue en Vercel). El video **no muestra funcionalidades del sistema de parqueadero**; por esta razón, las funcionalidades descritas se presentan como diseño propuesto.

1. Estructura de carpetas **EXISTENTE**

```
Parkinloock-/
├── .gitignore           Archivos excluidos de Git (node_modules, .env, dist, ...)
├── LICENSE             Licencia Apache 2.0
├── README.md           Descripción del proyecto
├── backend-Parkinlock/ Backend (vacío actualmente)
├── frontend-Parkinlock-/ Frontend PWA
│   ├── index.html      Página HTML principal (íconos, título "Parkinlock")
│   ├── package.json    Dependencias y scripts
│   ├── vite.config.js  Configuración de Vite, React y PWA (manifest)
│   ├── pwa-assets.config.js Configuración del generador de íconos
│   ├── .oxlintrc.json  Configuración del linter
│   ├── public/         Archivos estáticos e íconos PWA
│   │   ├── favicon.svg  Imagen base de los íconos
│   │   ├── favicon.ico
│   │   ├── pwa-64x64.png, pwa-192x192.png, pwa-512x512.png
│   │   ├── maskable-icon-512x512.png
│   │   └── apple-touch-icon-180x180.png
│   └── src/
│       ├── main.jsx     Punto de entrada: monta <App /> en #root
│       ├── App.jsx      Componente principal (plantilla de ejemplo)
│       ├── App.css, index.css Estilos
│       └── assets/      Imágenes
```

2. Archivos principales

Archivo	Función
<code>frontend-Parkinlock-/vite.config.js</code>	Registra los plugins de React y PWA. Define el manifest (nombre "Parkinlock", colores, modo standalone e íconos) y la actualización automática del service worker.
<code>frontend-Parkinlock-/pwa-assets.config.js</code>	Usa el preset <i>minimal-2023</i> para generar los íconos desde <code>public/favicon.svg</code> .
<code>frontend-Parkinlock-/src/main.jsx</code>	Inicializa React en modo estricto y renderiza el componente App.
<code>frontend-Parkinlock-/package.json</code>	Lista dependencias y scripts de ejecución.
<code>frontend-Parkinlock-/index.html</code>	Documento HTML con enlaces a íconos, color de tema y contenedor #root.

3. Tecnologías

Capa	Tecnología	Estado
Frontend	React 19.2, Vite 8.3, JavaScript (JSX)	EXISTENTE
PWA	vite-plugin-pwa 1.3 (Workbox), @vite-pwa/assets-generator 1.0.4	EXISTENTE
Calidad de código	oxlint 1.81	EXISTENTE
Control de versiones	Git + GitHub	EXISTENTE
Entorno de ejecución	Node.js + npm	EXISTENTE
Backend	[Pendiente de definir]	PROPUESTO
Base de datos	MySQL (Clever Cloud)	PROPUESTO
Despliegue frontend	Vercel	PROPUESTO
Editor	Cursor	Usado en el desarrollo

4. Funcionamiento técnico

Vite sirve la aplicación React durante el desarrollo con recarga instantánea. En producción, `npm run build` genera la carpeta `dist/` con los archivos optimizados; vite-plugin-pwa agrega `manifest.webmanifest`,

`registerSW.js` y `sw.js` (Workbox), que almacena en caché los recursos para cargas más rápidas e instalación como aplicación.

5. Conexión con la base de datos PROPUESTO

El frontend **nunca** se conecta directamente a MySQL. El backend se conectará usando variables de entorno definidas en un archivo `.env` (excluido de Git):

```
DB_HOST=      # host entregado por Clever Cloud
DB_PORT=3306
DB_USER=
DB_PASSWORD=
DB_NAME=parqueadero_db
```

En el frontend, la URL de la API se configurará con una variable de Vite, por ejemplo `VITE_API_URL` [Pendiente de completar].

6. Ejecución local

Requisitos: Node.js (versión LTS) y Git.

```
git clone https://github.com/camilotobon34-sudo/Parkinlock-.git
cd Parkinlock-/frontend-Parkinlock-
npm install
npm run dev      # abre http://localhost:5173
```

Script	Comando	Descripción
dev	<code>npm run dev</code>	Servidor de desarrollo.
build	<code>npm run build</code>	Compila para producción en dist/ (incluye service worker).
preview	<code>npm run preview</code>	Sirve localmente el build de producción.
lint	<code>npm run lint</code>	Analiza el código con oxlint.
generate-pwa-assets	<code>npm run generate-pwa-assets</code>	Regenera los íconos PWA desde public/favicon.svg.

Importante

Los comandos `npm` del frontend deben ejecutarse **dentro** de la carpeta `frontend-Parkinlock-`. Si se ejecutan en la raíz del repositorio, se crea un `package.json` incorrecto.

7. Configuración

- **Nombre y colores de la PWA:** en `vite.config.js`, sección `manifest` (actualmente color `#ffffff`; se recomienda usar el azul del logo).
- **Íconos:** reemplazar `public/favicon.svg` por el logo oficial y ejecutar `npm run generate-pwa-assets`.
- **Título de la página:** etiqueta `<title>` en `index.html`.

8. Despliegue

Sin despliegue registrado

No se encontró evidencia de despliegue en el repositorio. Siguiendo la metodología del curso, el procedimiento propuesto es: (1) importar el repositorio en Vercel, (2) seleccionar `frontend-Parkinlock-` como directorio raíz, (3) usar el comando de build `npm run build` y el directorio de salida `dist`. URL de producción: **[Pendiente de completar]**